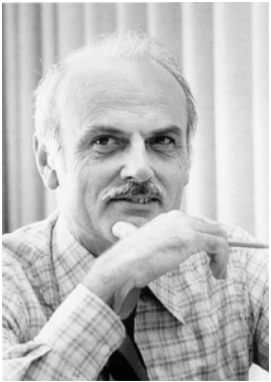
Kapitel 2

Das Relationale Modell

Relationales Modell

Überblick

- 1970 von Edgar F. Codd (IBM) eingeführt
- 1981 Turing Award („Nobelpreis der Informatik“) für diese Arbeit
- Grundlage für die meisten Datenbanksysteme (z.B. Oracle, Microsoft SQL Server, IBM DB2, Sybase, Postgres, MySQL)



Edgar F. Codd

Eigenschaften:

- Einfache & einheitliche Datenstrukturen – Relationen (Tabellen)
- Solide theoretische Grundlage (wichtig später für Anfrageverarbeitung & -optimierung)

Anwendung:

- **Direkt:** Erstellung von Relationen (Tabellen) mittels SQL-Befehlen
- **Abgeleitet:** aus einem gegebenen Entity-Relationship-Schema

[2] Codd – „A relational model of data for large shared data banks“, 1970.

Domäne

- Ist ein Wertebereich (oder Typ)
- Logische Menge von Werten

Beispiele

- D_1 = Ganzzahl ($\mathbb{Z}$)
- D_2 = Zeichenkette (String)
- D_3 = Datum
- D_4 = {rot, gelb, grün, blau}
- D_5 = {1, 2, 3}

Kann eine *begrenzte* oder *unbegrenzte* Kardinalität $|\ldots|$ haben:

- $|D_1| = \infty$ (Unendliche Menge: z.B. $\mathbb{Z}$)
- $|D_4| = 4$
- $|D_5| = 3$

Kartesisches Produkt

Das **kartesische Produkt** (auch: *Kreuzprodukt*) ist die Menge aller möglichen Kombinationen von Elementen aus k Mengen

Kartesisches Produkt

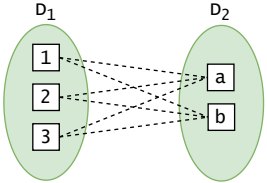
Das **kartesische Produkt** (auch: *Kreuzprodukt*) ist die Menge aller möglichen Kombinationen von Elementen aus k Mengen

Beispiel (k = 2)

$D_1 = \{1, 2, 3\}$

$D_2 = \{a, b\}$

$D_1 \times D_2 = \{(1, a), (1, b), (2, a), (2, b), (3, a), (3, b)\}$



Kartesisches Produkt

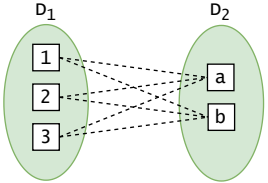
Das **kartesische Produkt** (auch: *Kreuzprodukt*) ist die Menge aller möglichen Kombinationen von Elementen aus k Mengen

Beispiel (k = 2)

$D_1 = \{1, 2, 3\}$

$D_2 = \{a, b\}$

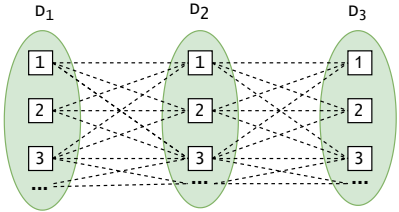
$D_1 \times D_2 = \{(1, a), (1, b), (2, a), (2, b), (3, a), (3, b)\}$



Beispiel (k = 3)

$D_1 = D_2 = D_3 = \mathbb{N}$

$D_1 \times D_2 \times D_3 = \{(1, 1, 1), (1, 1, 2), \ldots, (1, 2, 1), \ldots\}$



Relationales Modell

Relationen

Eine **Relation** R über einer Sammlung von Mengen (Domänen) $D_1, D_2, \cdots, D_k$ ist eine Teilmenge des kartesischen Produkts $D_1 \times D_2 \times \cdots \times D_k$

$$R \subseteq D_1 \times D_2 \times \cdots \times D_k$$

Relationales Modell

Relationen

Eine **Relation** R über einer Sammlung von Mengen (Domänen) $D_1, D_2, \cdots, D_k$ ist eine Teilmenge des kartesischen Produkts $D_1 \times D_2 \times \cdots \times D_k$

$$R \subseteq D_1 \times D_2 \times \cdots \times D_k$$

Beispiel (k = 2)

$D_1 = \{1, 2, 3\}, \quad D_2 = \{a, b\}$

$R_1 = \{\}$ *(leere Relation)*

$R_2 = \{(1, a), (2, b)\}$

$R_3 = \{(1, a), (2, a), (3, a)\}$

$R_4 = D_1 \times D_2 = \{(1, a), (1, b), (2, a), (2, b), (3, a), (3, b)\}$

Relationales Modell

Relationen

Eine **Relation** R über einer Sammlung von Mengen (Domänen) $D_1, D_2, \cdots, D_k$ ist eine Teilmenge des kartesischen Produkts $D_1 \times D_2 \times \cdots \times D_k$

$$R \subseteq D_1 \times D_2 \times \cdots \times D_k$$

Beispiel (k = 2)

$D_1 = \{1, 2, 3\}, \quad D_2 = \{a, b\}$

$R_1 = \{\}$ *(leere Relation)*

$R_2 = \{(1, a), (2, b)\}$

$R_3 = \{(1, a), (2, a), (3, a)\}$

$R_4 = D_1 \times D_2 = \{(1, a), (1, b), (2, a), (2, b), (3, a), (3, b)\}$

- *Endliche* und *unendliche* Relationen (wenn mindestens eine Domäne unendlich ist)
- In Datenbanksystemen: nur endliche Relationen (unendliche nicht realisierbar)
- **Relation** ist eine Menge von ***n*-Tupeln** $(d_1, d_2, \cdots, d_n)$ mit $d_i \in D_i$
- Domänen $D_1, \ldots, D_n$ als Spalten einer Tabelle interpretierbar

Grad (Stelligkeit)

Für Relation $R \subseteq D_1 \times \cdots \times D_k$ — k ist **Grad (Stelligkeit)** der Relation

Kardinalität

Anzahl der Tupel in einer Relation ($| R |$)

Attribut

Spalte in einer Relation (Tabelle)

Reihenfolge

Relation ist eine Menge von Tupeln → Reihenfolge der Tupel ist nicht definiert:

$$\{(0, a), (1, b)\} = \{(1, b), (0, a)\}$$

Reihenfolge der Attribute ist jedoch relevant:

$$\{(a, 0), (b, 1)\} \neq \{(0, a), (1, b)\}$$

Relationales Modell

Beispiel

```
StudId = {412, 307, 540}
StudName = {Smith, Jones}
Major = {CS, CSE, BIO}
```

r ist eine Relation über `StudId` $\times$ `StudName` $\times$ `Major`:

```
r = {(412, Smith, CS), (307, Jones, CSE), (412, Smith, CSE)}
```

Relationenschema

Seien $A_1, A_2, ..., A_k$ Attribute mit Domänen $D_1, D_2, ..., D_k$:

$$R(A_1 : D_1, A_2 : D_2, ..., A_k : D_k) \text{ ist } \textbf{Relationenschema}$$

– Legt den Namen und die Struktur der Relation fest

Beispiel

`Student(StudId: number, StudName: string, Major: string)`

Eine Sammlung von Relationen-Schemata = **relationales Datenbankschema**

Relationenschema

Geordnetes Relationenschema

- Relationenschema R ist ein k-Tupel von Domänen (Attributen)
- Attribute werden anhand ihrer Position im Tupel referenziert
- Zusätzlich können Attribute benannt werden

$$R = (A_1 : D_1, A_2 : D_2, \ldots, A_k : D_k)$$

Relationenschema

Geordnetes Relationenschema

- Relationenschema R ist ein k-Tupel von Domänen (Attributen)
- Attribute werden anhand ihrer Position im Tupel referenziert
- Zusätzlich können Attribute benannt werden

$$R = (A_1 : D_1, A_2 : D_2, \ldots, A_k : D_k)$$

Ungeordnetes Relationenschema

- Relationenschema R ist eine Menge von Attributen
- Jedem Attribut A_i ist eine Domäne D_i zugeordnet
- Attribute werden über ihre Namen referenziert

$$R = \{A_1, \ldots, A_k\} \quad \text{mit} \quad dom(A_i) = D_i,\ 1 \leqslant i \leqslant k$$

Relationenschema

Beispiel

Name	Einwohner	Bundesland
München	1.578.667	Bayern
Bremen	691.703	Bremen
Passau	51.907	Bayern

Relationenschema

Beispiel

Name	Einwohner	Bundesland
München	1.578.667	Bayern
Bremen	691.703	Bremen
Passau	51.907	Bayern

Geordnetes Relationenschema

Schema `R = (Name: String, Einwohner: Integer, Bundesland: String)`

Instanz `r = {('München', 1.578.667, 'Bayern'), ('Bremen', 691.703, 'Bremen'), ('Passau', 51.907, 'Bayern')}`

Relationenschema

Beispiel

Name	Einwohner	Bundesland
München	1.578.667	Bayern
Bremen	691.703	Bremen
Passau	51.907	Bayern

Geordnetes Relationenschema

Schema $R = (\text{Name}: \text{String}, \text{Einwohner}: \text{Integer}, \text{Bundesland}: \text{String})$

Instanz $r = \{(\text{'München'}, 1.578.667, \text{'Bayern'}), (\text{'Bremen'}, 691.703, \text{'Bremen'}), (\text{'Passau'}, 51.907, \text{'Bayern'})\}$

Ungeordnetes Relationenschema

Schema $R = \{\text{Name}, \text{Einwohner}, \text{Bundesland}\}$ mit $\text{dom}(\text{Name}) = \text{String}$, $\text{dom}(\text{Einwohner}) = \text{Integer}$, $\text{dom}(\text{Bundesland}) = \text{String}$

Instanz $t = \{t_1, t_2, t_3\}$ mit $t_1(\text{Name}) = \text{'München'}$, $t_1(\text{Einwohner}) = 1.578.667$, ...

Vorteil des geordneten Relationenschemas

– Kompaktere Schreibweise (z. B. beim Einfügen neuer Tupel)

$$t_3 = (\texttt{'Passau'}, 51.907, \texttt{'Bayern'})$$

vs.

$$t_3(\texttt{Name}) = \texttt{'Passau'}, t_3(\texttt{Einwohner}) = 51.907, \ldots$$

Relationenschema

Diskussion

Vorteil des geordneten Relationenschemas

– Kompaktere Schreibweise (z. B. beim Einfügen neuer Tupel)

$$t_3 = (\texttt{'Passau'}, 51.907, \texttt{'Bayern'})$$

vs.

$$t_3(\texttt{Name}) = \texttt{'Passau'}, t_3(\texttt{Einwohner}) = 51.907, \ldots$$

Nachteil des geordneten Relationenschemas

– Eingeschränkte logische Datenunabhängigkeit (z. B. neue Attribute können nur am Ende eingefügt werden)

Relationenschema

Diskussion

Vorteil des geordneten Relationenschemas

– Kompaktere Schreibweise (z. B. beim Einfügen neuer Tupel)

$$t_3 = (\texttt{'Passau'}, 51.907, \texttt{'Bayern'})$$

vs.

$$t_3(\texttt{Name}) = \texttt{'Passau'}, t_3(\texttt{Einwohner}) = 51.907, \ldots$$

Nachteil des geordneten Relationenschemas

– Eingeschränkte logische Datenunabhängigkeit (z. B. neue Attribute können nur am Ende eingefügt werden)

$\Rightarrow$ Beide Definitionen sind äquivalent

$\Rightarrow$ In diesem Kurs verwenden wir beide Definitionen

Relationeninstanz

oder kurz: Relation

- **Relationeninstanz** $r(R)$ eines Relationenschemas kann als Tabelle mit n Spalten und mehreren Zeilen verstanden werden
- Element $t \in r(R)$ wird als Tupel (oder Zeile) bezeichnet
- Instanz eines Datenbankschemas ist eine Sammlung von Relationen

Student

StudId	**StudName**	**Major**	← Relationenschema
123	Smith	CS	
235	Jones	CSE	← Tupel
367	Miller	CSE	

Relationeninstanz

oder kurz: Relation

- **Relationeninstanz** $r(R)$ eines Relationenschemas kann als Tabelle mit n Spalten und mehreren Zeilen verstanden werden
- Element $t \in r(R)$ wird als Tupel (oder Zeile) bezeichnet
- Instanz eines Datenbankschemas ist eine Sammlung von Relationen

Student

StudId	**StudName**	**Major**	← Relationenschema
123	Smith	CS	
235	Jones	CSE	← Tupel
367	Miller	CSE	

Eigenschaften einer Relation

- Die Reihenfolge der Zeilen (Tupel) ist irrelevant, z. B.:

$$\{(0, a), (1, b)\} = \{(1, b), (0, a)\}$$

Relationeninstanz

oder kurz: Relation

- **Relationeninstanz** $r(R)$ eines Relationenschemas kann als Tabelle mit n Spalten und mehreren Zeilen verstanden werden
- Element $t \in r(R)$ wird als Tupel (oder Zeile) bezeichnet
- Instanz eines Datenbankschemas ist eine Sammlung von Relationen

Student

StudId	**StudName**	**Major**	← Relationenschema
123	Smith	CS	
235	Jones	CSE	← Tupel
367	Miller	CSE	

Eigenschaften einer Relation

- Die Reihenfolge der Zeilen (Tupel) ist irrelevant, z. B.:

$$\{(0, a), (1, b)\} = \{(1, b), (0, a)\}$$

- Keine doppelten Zeilen (Tupel) in einer Relation, z. B.:

$$\{(0, a), (0, a), (0, a), (1, b)\} = \{(0, a), (1, b)\}$$

Integritätsbedingungen

Integritätsbedingungen in einem DBMS sind Regeln und Einschränkungen, die die *Korrektheit*, *Konsistenz* und *Gültigkeit* der Daten in einer Datenbank sicherstellen

- Müssen für jede (zulässige) Instanz eines Relationenschemas erfüllt sein
  - werden bei der Definition des Schemas angegeben
  - werden vom DBMS bei Änderungen an Relationen überprüft

Wenn das DBMS Integritätsbedingungen überprüft, entsprechen die verwalteten Daten besser der realen Welt, die modelliert werden soll!

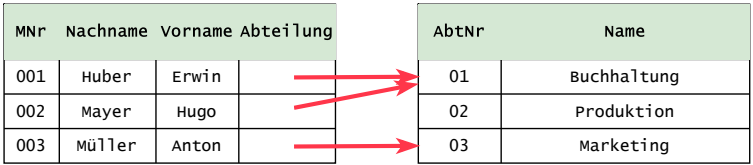
Beispiele

- Primär- und Fremdschlüssel-Bedingungen
- Eindeutigkeitsbedingungen (Uniqueness)
- `NOT NULL`-Bedingungen

Schlüssel

- Tupel müssen eindeutig identifizierbar sein (z. B. zur Referenzierung)
- Im relationalen Modell werden Schlüssel (eindeutige Identifikatoren) aus Attributen gebildet
- Konvention: Schlüsselfelder werden unterstrichen dargestellt

Beispiel



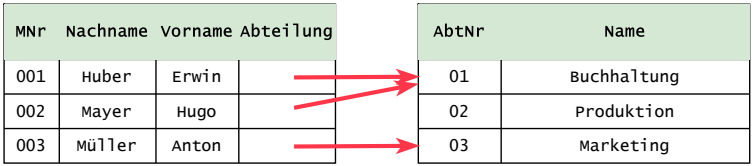
MNr	Nachname	Vorname	Abteilung
001	Huber	Erwin	
002	Mayer	Hugo	
003	Müller	Anton	

AbtNr	Name
01	Buchhaltung
02	Produktion
03	Marketing

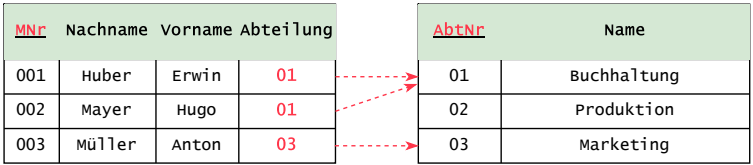
Schlüssel

- Tupel müssen eindeutig identifizierbar sein (z. B. zur Referenzierung)
- Im relationalen Modell werden Schlüssel (eindeutige Identifikatoren) aus Attributen gebildet
- Konvention: Schlüsselfelder werden unterstrichen dargestellt

Beispiel



MNr und AbtNr werden zu **Primärschlüsseln**



Schlüssel

Formale Notation

Eine Teilmenge S der Attribute eines Relationsschemas R ($S \subseteq R$) ist ein **Schlüssel**, wenn gilt:

Schlüssel

Formale Notation

Eine Teilmenge S der Attribute eines Relationsschemas R ($S \subseteq R$) ist ein **Schlüssel**, wenn gilt:

(1) **Eindeutigkeit** — Keine Instanz von R darf zwei Tupel enthalten, die in allen Attributen aus S übereinstimmen

Schlüssel

Formale Notation

Eine Teilmenge S der Attribute eines Relationsschemas R ($S \subseteq R$) ist ein **Schlüssel**, wenn gilt:

(1) **Eindeutigkeit** — Keine Instanz von R darf zwei Tupel enthalten, die in allen Attributen aus S übereinstimmen

(2) **Minimalität** — Es gibt keine echte Teilmenge $T \subset S$, die ebenfalls die **Eindeutigkeits**-Eigenschaft erfüllt

Schlüssel

Formale Notation

Eine Teilmenge S der Attribute eines Relationsschemas R ($S \subseteq R$) ist ein **Schlüssel**, wenn gilt:

(1) **Eindeutigkeit** — Keine Instanz von R darf zwei Tupel enthalten, die in allen Attributen aus S übereinstimmen

(2) **Minimalität** — Es gibt keine echte Teilmenge $T \subset S$, die ebenfalls die **Eindeutigkeits**-Eigenschaft erfüllt

Superschlüssel

- Menge $S \subseteq R$ ist ein **Superschlüssel**, wenn sie lediglich die **Eindeutigkeit** erfüllt
- Zum Beispiel ist die vollständige Attributmenge $S = R$ ein (trivialer) Superschlüssel für Relation R

Schlüssel

Beispiel

Id	KursNr	Titel	Semester
1	012	Einführung in die Informatik	WS 2023/24
2	012	Einführung in die Informatik	WS 2024/25
3	013	Medizinische Informationssysteme	WS 2024/25

Schlüssel

Beispiel

Id	KursNr	Titel	Semester
1	012	Einführung in die Informatik	WS 2023/24
2	012	Einführung in die Informatik	WS 2024/25
3	013	Medizinische Informationssysteme	WS 2024/25

`{Id}` Schlüssel (eindeutig, minimal)

Schlüssel

Beispiel

Id	KursNr	Titel	Semester
1	012	Einführung in die Informatik	WS 2023/24
2	012	Einführung in die Informatik	WS 2024/25
3	013	Medizinische Informationssysteme	WS 2024/25

`{Id}` Schlüssel (eindeutig, minimal)

`{KursNr}` kein Schlüssel (nicht eindeutig)

Schlüssel

Beispiel

Id	KursNr	Titel	Semester
1	012	Einführung in die Informatik	WS 2023/24
2	012	Einführung in die Informatik	WS 2024/25
3	013	Medizinische Informationssysteme	WS 2024/25

`{Id}`	Schlüssel (eindeutig, minimal)
`{KursNr}`	kein Schlüssel (nicht eindeutig)
`{Titel}`	kein Schlüssel (nicht eindeutig)

Schlüssel

Beispiel

Id	KursNr	Titel	Semester
1	012	Einführung in die Informatik	WS 2023/24
2	012	Einführung in die Informatik	WS 2024/25
3	013	Medizinische Informationssysteme	WS 2024/25

`{Id}`	Schlüssel (eindeutig, minimal)
`{KursNr}`	kein Schlüssel (nicht eindeutig)
`{Titel}`	kein Schlüssel (nicht eindeutig)
`{Semester}`	kein Schlüssel (nicht eindeutig)

Schlüssel

Beispiel

Id	KursNr	Titel	Semester
1	012	Einführung in die Informatik	WS 2023/24
2	012	Einführung in die Informatik	WS 2024/25
3	013	Medizinische Informationssysteme	WS 2024/25

`{Id}`	Schlüssel (eindeutig, minimal)
`{KursNr}`	kein Schlüssel (nicht eindeutig)
`{Titel}`	kein Schlüssel (nicht eindeutig)
`{Semester}`	kein Schlüssel (nicht eindeutig)
`{Id, KursNr}`	kein Schlüssel, aber Superschlüssel (eindeutig, nicht minimal)

Schlüssel

Beispiel

Id	KursNr	Titel	Semester
1	012	Einführung in die Informatik	WS 2023/24
2	012	Einführung in die Informatik	WS 2024/25
3	013	Medizinische Informationssysteme	WS 2024/25

`{Id}`	Schlüssel (eindeutig, minimal)
`{KursNr}`	kein Schlüssel (nicht eindeutig)
`{Titel}`	kein Schlüssel (nicht eindeutig)
`{Semester}`	kein Schlüssel (nicht eindeutig)
`{Id, KursNr}`	kein Schlüssel, aber Superschlüssel (eindeutig, nicht minimal)
`{KursNr, Semester}`	Schlüssel (eindeutig, minimal)

Zusammengesetzte Schlüssel

Oft reicht ein einzelnes Attribut nicht aus, um ein Tupel eindeutig zu identifizieren

Beispiel

KursNr	**Titel**	**Semester**
012	Einführung in die Informatik	WS 2023/24
012	Einführung in die Informatik	WS 2024/25
013	Medizinische Informationssysteme	WS 2024/25

Zusammengesetzte Schlüssel

Oft reicht ein einzelnes Attribut nicht aus, um ein Tupel eindeutig zu identifizieren

Beispiel

KursNr	**Titel**	**Semester**
012	Einführung in die Informatik	WS 2023/24
012	Einführung in die Informatik	WS 2024/25
013	Medizinische Informationssysteme	WS 2024/25

$\Rightarrow$ Zusammengesetzter Schlüssel: (`KursNr`, `Semester`)

Primärschlüssel

- Es ist wichtig, *alle* möglichen Schlüssel zu ermitteln ⇒ Menge der **Kandidatenschlüssel**
- Wenn es mehr als einen Schlüssel gibt, wird einer davon (vom Designer oder der Datenbankadministrator:in) als **Primärschlüssel** ausgewählt
- Für Kandidatenschlüssel, die nicht als Primärschlüssel gewählt wurden, können dennoch Eindeutigkeitsbedingungen definiert werden
- Hinweis: Es ist oft sinnvoll, einen künstlichen Primärschlüssel (ein einzelnes Attribut ⇒ Surrogatschlüssel) einzuführen, insbesondere wenn Relation häufig referenziert wird

Semantik von Schlüsselattributen

Eindeutigkeit bezieht sich nicht nur auf die aktuellen Tupel in der Relationsinstanz *r*

$\Rightarrow$ **Semantik** der Anwendung in der realen Welt

MitarbNr	Name	Gehalt
001	Müller	1700 €
002	Mayer	2172 €
003	Huber	3189 €
004	Schulz	2171 €

Im Beispiel wären `{MitarbNr}`, `{Name}` und `{Gehalt}` aktuell jeweils eindeutig — aber es könnten künftig Mitarbeitende mit gleichem Namen und/oder Gehalt auftreten.

Fremdschlüssel

Ein **Fremdschlüssel** ist eine Menge von Attributen in einer Relation (→ *Kind-Relation*), die verwendet wird, um auf ein Tupel in einer anderen Relation zu „verweisen“ (→ *Eltern-Relation*)

Fremdschlüssel

Ein **Fremdschlüssel** ist eine Menge von Attributen in einer Relation (→ *Kind-Relation*), die verwendet wird, um auf ein Tupel in einer anderen Relation zu „verweisen“ (→ *Eltern-Relation*)

Eigenschaften

- Muss auf den Primärschlüssel der referenzierten Relation verweisen
- Fremd- und Primärschlüsselattribute müssen übereinstimmende Domänen haben
- Notwendig in Relationsschemata, die aus Beziehungstypen abgeleitet wurden

Fremdschlüssel

Ein **Fremdschlüssel** ist eine Menge von Attributen in einer Relation (→ *Kind-Relation*), die verwendet wird, um auf ein Tupel in einer anderen Relation zu „verweisen“ (→ *Eltern-Relation*)

Eigenschaften

- Muss auf den Primärschlüssel der referenzierten Relation verweisen
- Fremd- und Primärschlüsselattribute müssen übereinstimmende Domänen haben
- Notwendig in Relationsschemata, die aus Beziehungstypen abgeleitet wurden

Beispiel

```
PRODUKTE(Produktname, ...)
LIEFERANTEN(Name, ...)
KUNDEN(Vorname, Nachname, ...)
bietet(Produktname → PRODUKTE, Name → LIEFERANTEN, Preis)
bestellt((Vorname, Nachname) → KUNDEN, Menge, Name → LIEFERANTEN, Produktname → PRODUKTE)
```

Fremdschlüsselbedingungen und referenzielle Integrität

- Eine **Fremdschlüsselbedingung** ist für ein Tupel erfüllt, wenn entweder
  - einige Werte der Fremdschlüsselattribute `NULL` sind (d. h. die Referenz ist unbekannt), oder
  - die Werte der Fremdschlüsselattribute als Primärschlüsselwerte eines Tupels in der Elternrelation vorkommen

Fremdschlüsselbedingungen und referenzielle Integrität

- Eine **Fremdschlüsselbedingung** ist für ein Tupel erfüllt, wenn entweder
  - einige Werte der Fremdschlüsselattribute `NULL` sind (d. h. die Referenz ist unbekannt), oder
  - die Werte der Fremdschlüsselattribute als Primärschlüsselwerte eines Tupels in der Elternrelation vorkommen
- Die Kombination der Fremdschlüsselattribute bildet im Relationenschema oft den Primärschlüssel der Relation, z. B.:

  `bietet(Produktname` $\longrightarrow$ `PRODUKTE, Name` $\longrightarrow$ `LIEFERANTEN, Preis)`

Fremdschlüsselbedingungen und referenzielle Integrität

- Eine **Fremdschlüsselbedingung** ist für ein Tupel erfüllt, wenn entweder
  - einige Werte der Fremdschlüsselattribute `NULL` sind (d. h. die Referenz ist unbekannt), oder
  - die Werte der Fremdschlüsselattribute als Primärschlüsselwerte eines Tupels in der Elternrelation vorkommen
- Die Kombination der Fremdschlüsselattribute bildet im Relationenschema oft den Primärschlüssel der Relation, z. B.:

  `bietet(Produktname` → `PRODUKTE, Name` → `LIEFERANTEN, Preis)`
- Wenn alle Fremdschlüsselbedingungen eingehalten werden, ist die **referenzielle Integrität** sichergestellt, d. h. es existieren keine *defekten Referenzen*

Data Definition Language (DDL)

- Ermöglicht die Definition nicht nur einer Menge von Relationen, sondern auch zusätzlicher Informationen zu jeder Relation, darunter:
  - das Schema einer Relation
  - die Domänen der Attribute
  - Integritätsbedingungen
  - die Menge der mit einer Relation verknüpften Indizes (später)
  - die physische Speicherstruktur einer Relation (später)

Tabellendefinition in SQL

Definition eines Relationsschemas

```
CREATE TABLE T (  -- T - Name der Relation
    <a1> <d1> <c1>, -- Definition des ersten Attributs
    <a2> <d2> <c2>,
    ...
    <ak> <dk> <ck>, -- Definition des k. Attributs
    <tc1>
)
```

a_i – Name des i-ten Attributs

d_i – Typ (Domäne) des i-ten Attributs

c_i – Optionale Attributbedingung für das i-te Attribut

tc_i – Optionale Tabellenbedingung (z.B. Primärschlüssel, Fremdschlüssel)

Dies erzeugt ein Relationenschema mit einer zunächst leeren Relation (Instanz)

Grundlegende Datentypen in SQL

`INTEGER` (auch: `INTEGER4`, `INT`)

`SMALLINT` (auch: `INTEGER2`)

`FLOAT` und `DOUBLE`

`DECIMAL(p,q)` und `NUMERIC(p,q)` mit p = Gesamtanzahl der Stellen und q = Anzahl der Nachkommastellen

`CHARACTER(n)`, `CHAR(n)`: Zeichenketten fester Länge n

`CHARACTER VARYING(n)`, `VARCHAR(n)`: Zeichenketten variabler Länge (bis maximal n)

`DATE`, `TIME`, `TIMESTAMP`: Für Datums- und Zeitangaben

- Es existieren verschiedene Varianten und Dialekte von Datentypen
- Komplexe Datentypen wie `XML`, `JSON`, `NDARRAY` sind (häufig) proprietäre Erweiterungen kommerzieller Datenbanksysteme

Definition einfacher Einschränkungen in SQL

Einfache Einschränkungen (Constraints) können direkt nach der Attributdefinition angegeben werden

Beispiele

`NOT NULL` – Attributwert darf nicht *undefiniert* sein (in SQL: `NULL`)

`PRIMARY KEY` – Attribut ist Primärschlüssel (für nicht-zusammengesetzte Schlüssel)

`UNIQUE` – Attribut ist Kandidatenschlüssel

`DEFAULT` w_1 – Wert w_1 wird als Standard verwendet (sofern kein anderer Wert angegeben wird)

`CHECK` f – Formel f wird bei jeder Einfügung geprüft (z. B. `CHECK A` $\leqslant 100$)

`REFERENCES` $t_1(a_1)$ – Verweis auf Attribut a_1 in Tabelle t_1 (Fremdschlüssel)

Standardwerte (DEFAULT)

- Wenn bei einer `INSERT`-Anweisung kein Attributwert angegeben wird, kann ein `DEFAULT`-Wert verwendet werden
- Wenn kein Standardwert definiert ist, wird automatisch `NULL` verwendet

Beispiel

```
CREATE TABLE Assistent (
    PersNr INTEGER PRIMARY KEY,
    Name VARCHAR(80) NOT NULL,
    Abteilung VARCHAR(200) DEFAULT 'Informatik'
);
```

Integritätsbedingungen in SQL

Zusätzliche Integritätsbedingungen können unabhängig von der Attributdefinition angegeben werden

Beispiele

`PRIMARY KEY`$(A_1, A_2, ...)$ – zusammengesetzter Primärschlüssel

`UNIQUE`$(A_1, A_2, ...)$ – zusammengesetzter Kandidatenschlüssel

`FOREIGN KEY`$(A_1, A_2, ...)$ `REFERENCES` $t_1(B_1, B_2, ...)$ Verweis auf zusammengesetzten Schlüssel in Relation t_1

(Hinweis: Falls $(B_1, B_2, ...)$ weggelassen wird, wird angenommen, dass $(A_1, A_2, ...)$ verwendet wird)

`CHECK` f – Formel f wird bei jedem Einfügen geprüft (z. B. `CHECK A` $\leqslant 100$)

Hinweis: SQL unterscheidet nicht zwischen Groß- und Kleinschreibung (*Schlüsselwörter & Bezeichner sind standardmäßig nicht case-sensitive*)

Schlüsseldefinitionen in SQL

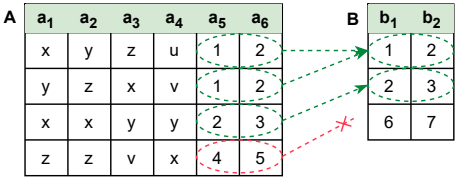
```
CREATE TABLE A (a1 VARCHAR(1),
                a2 VARCHAR(1),
                a3 VARCHAR(1),
                a4 VARCHAR(1),
                a5 INTEGER,
                a6 INTEGER,
                PRIMARY KEY(a1, a2),
                UNIQUE(a3, a4),
    FOREIGN KEY (a5, a6) REFERENCES B(b1, b2));
```

```
CREATE TABLE B (b1 INTEGER,
                b2 INTEGER,
                PRIMARY KEY(b1, b2));
```

Schlüsseldefinitionen in SQL

```
CREATE TABLE A (a1 VARCHAR(1),
                a2 VARCHAR(1),
                a3 VARCHAR(1),
                a4 VARCHAR(1),
                a5 INTEGER,
                a6 INTEGER,
                PRIMARY KEY(a1, a2),
                UNIQUE(a3, a4),
    FOREIGN KEY (a5, a6) REFERENCES B(b1, b2));
```

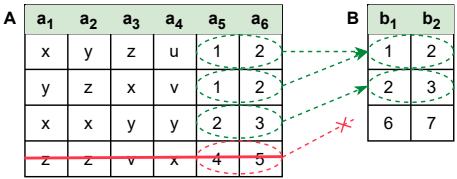
```
CREATE TABLE B (b1 INTEGER,
                b2 INTEGER,
                PRIMARY KEY(b1, b2));
```



Schlüsseldefinitionen in SQL

```
CREATE TABLE A (a1 VARCHAR(1),
                a2 VARCHAR(1),
                a3 VARCHAR(1),
                a4 VARCHAR(1),
                a5 INTEGER,
                a6 INTEGER,
                PRIMARY KEY(a1, a2),
                UNIQUE(a3, a4),
    FOREIGN KEY (a5, a6) REFERENCES B(b1, b2));
```

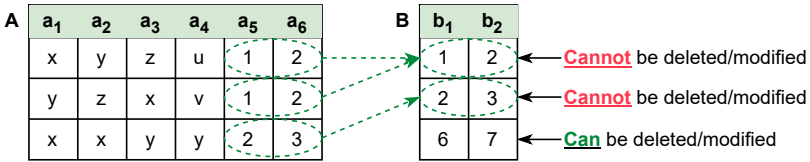
```
CREATE TABLE B (b1 INTEGER,
                b2 INTEGER,
                PRIMARY KEY(b1, b2));
```



- Tupel in Relation A ohne gültigen Partner in Relation B sind nicht zulässig
- Ohne zusätzliche Angaben können Tupel in B, die von Tupeln in A referenziert werden, weder gelöscht noch deren Werte b_1, b_2 verändert werden

Schlüsseldefinitionen in SQL

Das Löschen eines Tupels in B ist nicht möglich, wenn darauf referenziert wird



Alternativen

`FOREIGN KEY`(a_5, a_6) `REFERENCES` $B(b_1, b_2)$ `ON DELETE CASCADE`

Löschen eines Tupels in B führt zum Löschen aller referenzierenden Tupel in A

`FOREIGN KEY`(a_5, a_6) `REFERENCES` $B(b_1, b_2)$ `ON UPDATE CASCADE`

Änderung eines Tupels in B führt zur automatischen Änderung der referenzierenden Tupel in A

`FOREIGN KEY`(a_5, a_6) `REFERENCES` $B(b_1, b_2)$ `ON DELETE SET NULL`

„Herrenlose Referenzen“ in A werden auf `NULL` gesetzt

Tabellendefinition in SQL

Beispiele

Zusammengesetzter Primärschlüssel { kursname, semester }

```
CREATE TABLE Kurse (
    kursname VARCHAR(50) NOT NULL,
    semester VARCHAR(50) NOT NULL,
    ects INT CHECK (ects > 0) DEFAULT 6,
    dozent VARCHAR(100) NOT NULL,
    PRIMARY KEY (kursname,semester)
);
```

Einfacher Primärschlüssel { kursID }

```
CREATE TABLE Kurse (
    kursID INTEGER PRIMARY KEY,
    kursname VARCHAR(50) NOT NULL,
    semester VARCHAR(50) NOT NULL,
    ects INT CHECK (ects > 0) DEFAULT 6,
    dozent VARCHAR(100) NOT NULL
);
```

Tabellendefinition in SQL

Beispiel

Tabelle für Dozent:innen

```
CREATE TABLE lecturer (
    l_id INTEGER PRIMARY KEY,
    name VARCHAR(50),
    birthday DATE);
```

Tabelle für Beziehung zwischen Dozent:innen und Kursen

```
CREATE TABLE teaches (
    lecturer INTEGER REFERENCES lecturer(l_id) ON DELETE CASCADE,
    courseId INTEGER NOT NULL,
    semester VARCHAR(20) NOT NULL,
    PRIMARY KEY(lecturer, courseId, semester),
    FOREIGN KEY (courseId, semester) REFERENCES course);
```

Schlüsselwort `ON DELETE CASCADE`: Wird ein `lecturer`-Tupel gelöscht, werden auch alle zugehörigen (referenzierenden) Tupel in der `teaches`-Relation automatisch gelöscht

Ändern von Tabellendefinitionen in SQL

Löschen einer Tabelle

```
-- Das Relationsschema t1 und alle Tupel werden geloescht
DROP TABLE t1
```

Hinzufügen neuer Attribute

```
-- Neue Attribute werden rechts an die Tabelle angehängt
-- Für bereits existierende Tupel ist der Wert standardmäßig null
ALTER TABLE t1 ADD (<a1> <d1> <c1>, ...)
```

Löschen von Attributen aus einer Tabelle

```
ALTER TABLE t1 DROP (<a1>, <a2>, ...)
```

Ändern von Attributdefinitionen in einer Tabelle

```
ALTER TABLE t1 MODIFY (<a1> <d1> <c1>, ...)
```

Komplexes Tabellenbeispiel in PostgreSQL

```
CREATE TABLE Students (
    StID NUMERIC CONSTRAINT Students_pk PRIMARY KEY,
    FName VARCHAR(50) NOT NULL,
    LName VARCHAR(50) NOT NULL,
    DOB DATE CONSTRAINT dob_check
    CHECK(DOB IS NOT NULL AND to_char(DOB) > '01-JAN-01'),
    Major CHAR(5) CONSTRAINT fk_majors REFERENCES Majors,
    ZipCode INTEGER CONSTRAINT check_zip
        CHECK(ZipCode IS NOT NULL AND ZipCode BETWEEN 1 AND 99999),
    City VARCHAR(50),
    Street VARCHAR(50),
    Started DATE NOT NULL,
    CONSTRAINT dates_check CHECK(DOB < Started),
    CONSTRAINT name_add UNIQUE(FName, LName, DOB)
);
```

Zusammenfassung

- **Relationales Modell** bildet theoretische Grundlage relationaler Datenbanksysteme (RDBMS)
- Operationen auf Relationen folgen der *Mengensemantik* – d. h. keine feste Reihenfolge und keine Duplikate (Details später)
- SQL-DDL-Anweisungen ermöglichen die Definition mehrerer Relationen im Rahmen eines relationalen Datenbankschemas